1924 2026

COLEGIO ARRIARÁN BARROS

FUNDACION EDUCACIONAL ARRIARAN BARROS

PENSAMIENTO COMPUTACIONAL Y PROGRAMACIÓN

Práctica de pandas: Series y DataFrames

Se ejecuta en Python (IDLE) · 70 minutos

Nombre:	**Curso:**	**Fecha:**

Cómo trabajar

1. Abre **IDLE** → File → New File y guarda como `pandas_apellido_nombre.py` en una carpeta que puedas encontrar.
2. Escribe el código de cada recuadro en tu archivo, ejecútalo con **F5** y anota en esta hoja lo que muestra. No hace falta predecir antes.
3. Si algo sale en rojo, lee la última línea del error: dice qué pasó. Anótala y pide ayuda si no avanzas.
4. Todo el código de un ejercicio va en el mismo archivo, uno debajo de otro.

Ejercicio 0 — Preparación: comprobar que pandas funciona — ~10 min

a) Escribe estas dos líneas en un archivo nuevo y ejecútalo (**F5**):

```
import pandas as pd
print(pd.__version__)
```

b) Si aparece un número de versión, sigue al Ejercicio 1. Si aparece `ModuleNotFoundError`, abre el símbolo del sistema (tecla Windows, escribe `cmd`) y ejecuta `pip install pandas`. Cuando termine, vuelve a ejecutar tu archivo.

Lo que apareció	Versión o error
Resultado del print:	

Ejercicio 1 — Series: una columna con etiquetas

~12 min

a) Escribe y ejecuta este código. Anota lo que muestra cada `print`.

```
import pandas as pd

notas = pd.Series([6.5, 5.0, 4.5])
print(notas)
print(notas.dtype)
```

Línea	Lo que mostró
`print(notas)`	
`print(notas.dtype)`	

b) Ahora agrega etiquetas propias. Cambia la línea de la Series por esta y ejecuta:

```
notas = pd.Series([6.5, 5.0, 4.5], index=["Ana", "Beto", "Caro"])
print(notas)
print(notas["Beto"])
print(list(notas.index))
```

Línea	Lo que mostró
`print(notas)`	
`print(notas["Beto"])`	
`print(list(notas.index))`	

c) Compara a) y b): ¿qué cambió en la primera columna que se imprime? ¿Para qué sirve el índice?

Mi respuesta:

Ejercicio 2 — DataFrame desde un diccionario

~14 min

a) Escribe y ejecuta. Anota lo que muestra.

```
datos = {
    "nombre": ["Ana", "Beto", "Caro"],
    "edad": [16, 17, 16],
    "nota": [6.5, 5.0, 4.5]
}
df = pd.DataFrame(datos)

print(df)
print(df.shape)
print(list(df.columns))
print(type(df["nota"]).__name__)
```

Línea	Lo que mostró
`print(df)`	
`print(df.shape)`	
`print(list(df.columns))`	
`print(type(df["nota"]).__name__)`	

b) Ahora un diccionario con una columna más (`"curso"`). Escríbelo debajo de lo anterior, ejecuta y anota el resultado de `df_b.shape`:

```
datos_b = {
    "nombre": ["Ana", "Beto", "Caro"],
    "edad": [16, 17, 16],
    "nota": [6.5, 5.0, 4.5],
    "curso": ["3A", "3A", "3B"]
}
df_b = pd.DataFrame(datos_b)
print(df_b.shape)
```

Mi resultado:

c) Ahora la columna `"curso"` con una lista más corta. Python mostrará un error: anota su última línea y explica por qué falla. Después escribe un `#` al comienzo de la línea `df_c = ...` para que el resto del archivo siga funcionando.

```
datos_c = {
    "nombre": ["Ana", "Beto", "Caro"],
    "edad": [16, 17, 16],
    "nota": [6.5, 5.0, 4.5],
    "curso": ["3A", "3B"]
}
df_c = pd.DataFrame(datos_c)
```

Error y mi explicación:

Ejercicio 3 — DataFrame desde una lista de listas

~12 min

a) Cada sublista es una fila. Escribe, ejecuta y anota:

```
filas = [
    ["Ana", 16, 6.5],
    ["Beto", 17, 5.0],
    ["Caro", 16, 4.5]
]
df2 = pd.DataFrame(filas, columns=["nombre", "edad", "nota"])

print(df2)
print(df.equals(df2))
```

Línea	Lo que mostró
`print(df2)`	
`print(df.equals(df2))`	

b) Crea la tabla sin indicar los nombres de las columnas y anota lo que muestra:

```
df3 = pd.DataFrame(filas)
print(list(df3.columns))
```

Mi resultado:

c) Con tus palabras: ¿para qué sirve `columns=` cuando se crea una tabla desde una lista de listas?

Mi respuesta:

Ejercicio 4 — Guardar y leer un archivo CSV

~16 min

a) Guarda la tabla en un archivo y léelo de nuevo. Escribe, ejecuta y anota:

```
df = pd.DataFrame(datos)
df.to_csv("notas.csv", index=False)

leido = pd.read_csv("notas.csv")
print(leido)
print(leido["edad"].dtype)
print(leido["nota"].dtype)
```

Línea	Lo que mostró
`print(leido)`	
`print(leido["edad"].dtype)`	
`print(leido["nota"].dtype)`	

b) Busca el archivo `notas.csv` en la carpeta donde guardaste tu programa y ábrelo con el Bloc de notas. Copia aquí las primeras líneas:

Lo que vi dentro de notas.csv:

c) Guarda otra vez, pero sin `index=False`, y anota las columnas que aparecen al leerlo:

```
df.to_csv("notas2.csv")
otro = pd.read_csv("notas2.csv")
print(list(otro.columns))
```

Columnas que aparecieron:

d) ¿Qué columna apareció que no estaba antes y por qué crees que apareció?

Mi respuesta:

Ejercicio 5 — Mi propia tabla — ~6 min

a) Escribe, ejecuta y anota:

```
mi_tabla = pd.DataFrame({
    "producto": ["pan", "leche", "queso", "arroz"],
    "stock": [20, 15, 8, 30],
    "precio": [1200.0, 1100.0, 3500.0, 2000.0]
})
print(mi_tabla)
print(mi_tabla.shape)
print(mi_tabla.dtypes)
```

Línea	Lo que mostró
`print(mi_tabla)`	
`print(mi_tabla.shape)`	
`print(mi_tabla.dtypes)`	

b) Tu tabla tiene columnas de distintos tipos. Completa según lo que mostró `dtypes`:

Columna	Tipo que mostró	¿Texto, entero o decimal?
producto		
stock		
precio		

c) Crea tu propia tabla con 4 filas y 3 columnas sobre algo que te guste (música, deportes, series…). Debe incluir una columna de texto, una de números enteros y una de decimales. Escríbela en tu archivo, ejecútala y muéstrasela al profesor.

Antes de terminar

- ☐ Mi archivo está guardado con mi apellido y nombre.
- ☐ Entiendo qué es una Series y qué es una columna de un DataFrame.
- ☐ Sé crear un DataFrame desde un diccionario y desde una lista de listas.
- ☐ Sé guardar y leer un CSV, y sé qué hace `index=False`.

Mi registro de errores: escribe un error que te salió hoy y cómo lo solucionaste.

Error:	*Cómo lo solucioné:*